

Intelligent Near-Earth Object Hazard Prediction Using Machine Learning Machine Learning Internship Project Report

Submitted By

Name: Mohammad Khwaja Moinuddin Chisty

Internship

Machine Learning Internship

Organization

Internship Studio

ACKNOWLEDGEMENT

I would like to express my sincere appreciation to the Machine Learning Internship Program for offering a self-paced learning opportunity through its Learning Management System (LMS). The comprehensive course materials, practical exercises, and well-organized project instructions provided an excellent platform for strengthening my understanding of machine learning concepts and their real-world applications.

Working on the project, "Intelligent Near-Earth Object Hazard Prediction Using Machine Learning," has been a valuable learning experience that enabled me to bridge theoretical knowledge with practical implementation. Throughout this project, I gained hands-on experience in data preprocessing, exploratory data analysis, feature engineering, supervised learning techniques, model performance evaluation, explainable artificial intelligence, and web-based deployment using modern machine learning tools.

I would also like to extend my gratitude to NASA for making the Near-Earth Object (NEO) dataset publicly accessible. The availability of this dataset made it possible to develop, train, and evaluate a reliable machine learning model for hazardous asteroid prediction.

My sincere thanks go to the open-source developer community for providing powerful Python libraries such as NumPy, Pandas, Matplotlib, Seaborn, Scikit-learn, XGBoost, SHAP, Joblib, and Streamlit. These tools, along with their extensive documentation, greatly simplified the implementation and successful completion of this project.

Finally, I would like to thank my family and friends for their continuous encouragement, patience, and unwavering support throughout this internship journey. Their motivation inspired me to remain dedicated and successfully accomplish every stage of the project.

This internship has significantly enhanced my practical knowledge of machine learning and strengthened my analytical and problem-solving skills. The experience gained while developing this end-to-end prediction system has increased my confidence in applying machine learning techniques to real-world challenges and will serve as a strong foundation for my future academic and professional endeavors.

Contents

S.No	Content	Page No.
1	Project Overview	1-2
2	Problem Statement	3
3	Project Objectives	4
4	Dataset Description	5-7
5	Project Methodology	7-9
6	Data Preprocessing	9-19
7	Exploratory Data Analysis	20-24
8	Feature Engineering	25-26
9	Machine Learning Model Development	27-31
10	Model Evaluation	32-33
11	Explainable Artificial Intelligence	34-35
12	Model Deployment	36-37
13	Results	38
14	Conclusion	38
15	Future Scope	39
16	References	39

Project Overview

Introduction

Near-Earth Objects (NEOs) are celestial bodies, mainly asteroids and comets, whose orbital paths pass close to Earth's orbit. Although the majority of these objects pose no immediate threat, some are classified as Potentially Hazardous Asteroids (PHAs) because of their large size and the possibility of approaching Earth at relatively short distances. Continuous observation and accurate identification of these objects are essential for improving planetary defense strategies and minimizing potential risks.

Space agencies such as NASA regularly monitor and record information about thousands of Near-Earth Objects using advanced observation technologies. The collected data contains valuable physical and orbital attributes, including estimated diameter, relative velocity, miss distance, absolute magnitude, and orbital information. As the volume of observations continues to grow, manually examining every asteroid becomes increasingly difficult, making automated prediction techniques a practical necessity.

Machine learning offers an effective solution by learning complex relationships from historical asteroid observations and automatically identifying whether an asteroid is hazardous. Instead of relying solely on manual analysis, predictive models can process large datasets efficiently and provide faster, more consistent classification results.

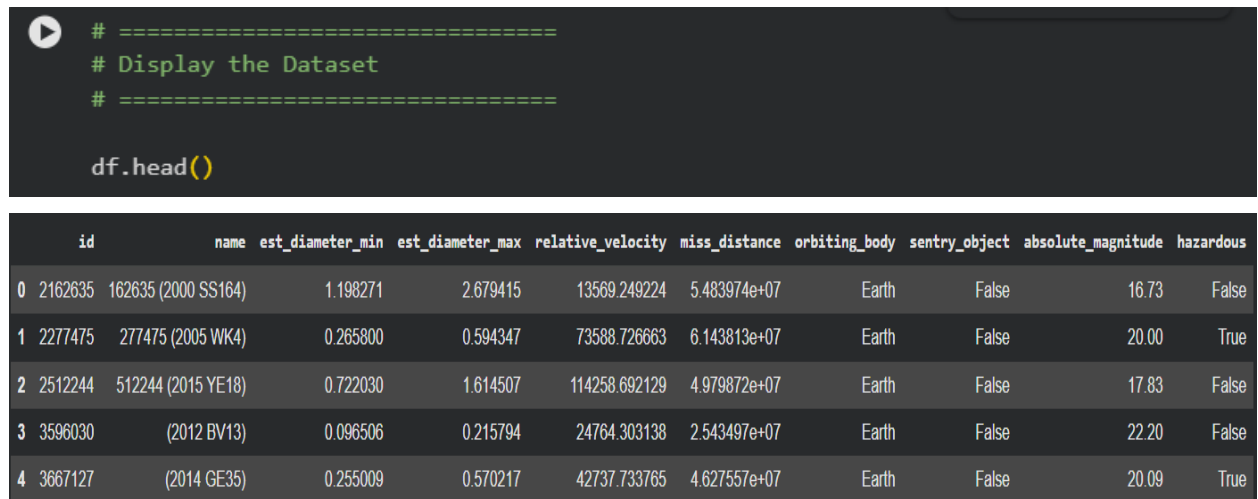
This project, "Intelligent Near-Earth Object Hazard Prediction Using Machine Learning," focuses on building a complete machine learning pipeline capable of predicting hazardous asteroids using NASA's Near-Earth Object dataset. The implementation includes data preprocessing, exploratory data analysis, feature engineering, balancing imbalanced classes using SMOTE, training multiple classification algorithms, evaluating their performance, and selecting the most suitable model for deployment.

To determine the best-performing algorithm, several supervised learning models, including Logistic Regression, Decision Tree, Random Forest, Gradient Boosting, and XGBoost, were developed and compared using evaluation metrics such as Accuracy, Precision, Recall, F1-Score, and ROC-AUC. Among these algorithms, the Random Forest Classifier produced the most reliable and balanced performance, making it the preferred model for the final prediction system.

In addition, Explainable Artificial Intelligence (XAI) techniques, including Feature Importance analysis and SHAP (SHapley Additive Explanations), were applied to improve model interpretability and understand the influence of individual features on predictions.

Finally, the selected model was integrated into a Streamlit web application that allows users to enter asteroid-related parameters and obtain instant predictions indicating whether an asteroid is hazardous or non-hazardous. This project demonstrates how machine learning can be applied to real-world astronomical datasets to support intelligent decision-making, automate hazard assessment, and contribute to future planetary defense research.

Figure 1: Preview of the Near Earth Object (NEO) Dataset



	id	name	est_diameter_min	est_diameter_max	relative_velocity	miss_distance	orbiting_body	sentry_object	absolute_magnitude	hazardous
0	2162635	162635 (2000 SS164)	1.198271	2.679415	13569.249224	5.483974e+07	Earth	False	16.73	False
1	2277475	277475 (2005 WK4)	0.265800	0.594347	73588.726663	6.143813e+07	Earth	False	20.00	True
2	2512244	512244 (2015 YE18)	0.722030	1.614507	114258.692129	4.979872e+07	Earth	False	17.83	False
3	3596030	(2012 BV13)	0.096506	0.215794	24764.303138	2.543497e+07	Earth	False	22.20	False
4	3667127	(2014 GE35)	0.255009	0.570217	42737.733765	4.627557e+07	Earth	False	20.09	True

PROBLEM STATEMENT

Introduction to the Problem:

Near-Earth Objects (NEOs), which include asteroids and comets, frequently travel through space on orbits that pass close to Earth. While the majority of these celestial objects do not pose any immediate danger, a small number possess physical and orbital characteristics that classify them as potentially hazardous. Parameters such as diameter, relative velocity, miss distance, and orbital path play an important role in determining the likelihood of a potential impact with Earth.

To ensure planetary safety, NASA continuously observes and records thousands of Near-Earth Objects using advanced tracking systems and space-based observation technologies. These observations generate large volumes of data describing the physical and orbital properties of asteroids. As the number of detected objects increases every year, manually examining and classifying each observation becomes increasingly complex, resource-intensive, and susceptible to delays.

Conventional approaches for identifying hazardous asteroids depend heavily on continuous monitoring and detailed scientific analysis. Although these methods are reliable, they are not well suited for processing massive datasets within a short period. Therefore, there is a growing demand for intelligent computational techniques capable of automatically analyzing asteroid characteristics and providing rapid hazard predictions.

Machine learning offers an efficient alternative by extracting meaningful patterns from historical asteroid observations and using them to classify newly detected objects. Such predictive models can support astronomers by improving the speed and consistency of hazard assessment while assisting in early warning and planetary defense planning.

To address this challenge, this project develops a supervised machine learning model that predicts whether a Near-Earth Object should be classified as Hazardous or Non-Hazardous using NASA's historical asteroid dataset. The proposed solution combines data preprocessing, machine learning algorithms, and Explainable Artificial Intelligence (XAI) techniques to deliver accurate, reliable, and interpretable predictions for real-world asteroid hazard assessment.

PROJECT OBJECTIVES

The main objective of this project is to build, evaluate, and deploy a machine learning-based prediction system that can accurately determine whether a Near-Earth Object (NEO) is hazardous by analyzing its physical and orbital properties.

The specific objectives of this project are:

- To study the NASA Near-Earth Object dataset and understand the significance of the available asteroid features.
- To prepare the dataset for model development through data cleaning, missing value verification, duplicate removal, categorical feature encoding, and feature standardization.
- To perform Exploratory Data Analysis (EDA) in order to examine feature distributions, identify trends, detect outliers, and analyze relationships between variables.
- To overcome class imbalance by applying the Synthetic Minority Over-sampling Technique (SMOTE), thereby improving the learning capability of classification models.
- To implement and compare different supervised machine learning algorithms such as Logistic Regression, Decision Tree, Random Forest, Gradient Boosting, and XGBoost for hazardous asteroid classification.
- To assess the performance of each model using evaluation measures including Accuracy, Precision, Recall, F1-Score, and ROC-AUC.
- To select the most effective model based on a comprehensive comparison of performance metrics and use it as the final prediction model.
- To enhance model interpretability by applying Explainable Artificial Intelligence (XAI) techniques, including Feature Importance analysis and SHAP (SHapley Additive Explanations).
- To deploy the optimized machine learning model as an interactive Streamlit application that allows users to perform real-time hazardous asteroid predictions through a simple web interface.

Expected Outcome:

The expected outcome of this project is the development of a robust and efficient machine learning system capable of accurately distinguishing hazardous asteroids from non-hazardous ones using historical NASA data. The final deployed application will enable users to provide asteroid characteristics and instantly receive prediction results, demonstrating the practical use of machine learning, explainable AI, and web deployment technologies in space research and planetary defense.

DATASET DESCRIPTION

1. Dataset Source

The dataset used in this project is the **Near-Earth Object (NEO) Dataset**, which contains historical observations of asteroids and other celestial objects that pass close to Earth's orbit. The dataset is derived from NASA's publicly available Near-Earth Object records and provides valuable information about the physical and orbital characteristics of each asteroid.

The dataset consists of multiple numerical and categorical attributes that describe an asteroid's estimated size, velocity, miss distance, brightness, orbiting body, and hazard classification. These features are used to train machine learning models capable of predicting whether a newly observed asteroid is hazardous or non-hazardous.

The target variable in the dataset is **hazardous**, which indicates whether an asteroid is considered potentially hazardous based on NASA's criteria.

2. Dataset Features

The following table describes the features used in this project after preprocessing.

Feature Name	Description
est_diameter_min	Minimum estimated diameter of the asteroid (kilometers)
est_diameter_max	Maximum estimated diameter of the asteroid (kilometers)
relative_velocity	Relative velocity of the asteroid with respect to Earth
miss_distance	Closest distance between the asteroid and Earth during its approach
orbiting_body	Celestial body around which the asteroid is orbiting
sentry_object	Indicates whether the asteroid is monitored by NASA's Sentry system
absolute_magnitude	Brightness of the asteroid
hazardous	Target variable indicating whether the asteroid is hazardous

3. Dataset Characteristics

The dataset contains historical observations collected from astronomical monitoring systems. Before model development, the dataset was analyzed to understand its structure, identify missing values, examine data types, and compute statistical summaries.

A detailed inspection of the dataset was performed using Pandas functions such as `head()`, `info()`, and `describe()`. These analyses provided valuable insights into the quality and distribution of the data before preprocessing.

Figure 2: Dataset information showing the number of records, feature names, data types, and non-null values.

```
# =====
# Dataset Information
# =====

df.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 90836 entries, 0 to 90835
Data columns (total 10 columns):
#   Column                Non-Null Count  Dtype
---  -
0   id                    90836 non-null  int64
1   name                  90836 non-null  object
2   est_diameter_min      90836 non-null  float64
3   est_diameter_max      90836 non-null  float64
4   relative_velocity     90836 non-null  float64
5   miss_distance         90836 non-null  float64
6   orbiting_body         90836 non-null  object
7   sentry_object         90836 non-null  bool
8   absolute_magnitude    90836 non-null  float64
9   hazardous            90836 non-null  bool
dtypes: bool(2), float64(5), int64(1), object(2)
memory usage: 5.7+ MB
```

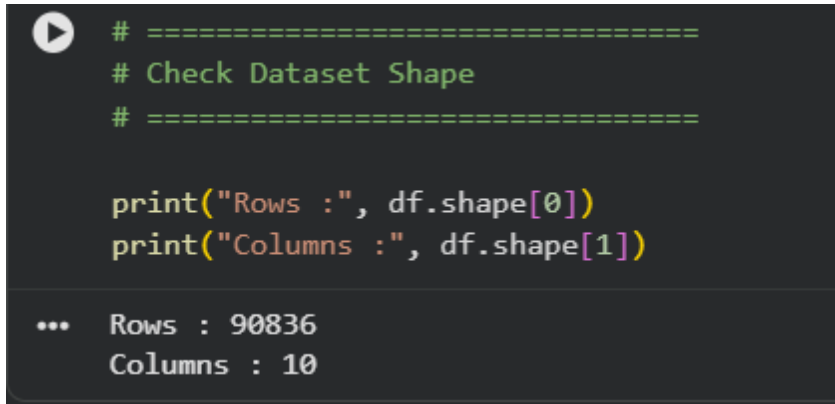
Figure 3: Statistical summary of the numerical features in the Near-Earth Object dataset.

```
# =====
# Summary Statistics
# =====

df.describe().T
```

	count	mean	std	min	25%	50%	75%	max
id	90836.0	1.438288e+07	2.087202e+07	2.000433e+06	3.448110e+06	3.748362e+06	3.884023e+06	5.427591e+07
est_diameter_min	90836.0	1.274321e-01	2.985112e-01	6.089126e-04	1.925551e-02	4.836765e-02	1.434019e-01	3.789265e+01
est_diameter_max	90836.0	2.849469e-01	6.674914e-01	1.361570e-03	4.305662e-02	1.081534e-01	3.206564e-01	8.473054e+01
relative_velocity	90836.0	4.806692e+04	2.529330e+04	2.033464e+02	2.861902e+04	4.419012e+04	6.292360e+04	2.369901e+05
miss_distance	90836.0	3.706655e+07	2.235204e+07	6.745533e+03	1.721082e+07	3.784658e+07	5.654900e+07	7.479865e+07
absolute_magnitude	90836.0	2.352710e+01	2.894086e+00	9.230000e+00	2.134000e+01	2.370000e+01	2.570000e+01	3.320000e+01

Figure 4: Shape of the dataset indicating the total number of observations and features available for analysis.



```
# =====  
# Check Dataset Shape  
# =====  
  
print("Rows :", df.shape[0])  
print("Columns :", df.shape[1])  
  
... Rows : 90836  
    Columns : 10
```

PROJECT METHODOLOGY

Overview:

The development of the Hazardous Asteroid Prediction System follows a well-defined machine learning methodology that converts raw astronomical observations into an intelligent classification model. Every stage of the workflow plays an important role in improving data quality, enhancing prediction accuracy, and ensuring the reliability of the final system.

The methodology adopted in this project consists of the following phases:

- 1.Dataset Collection
- 2.Data Preparation
- 3.Exploratory Data Analysis (EDA)
- 4.Feature Engineering
- 5.Feature Scaling
- 6.Data Splitting
- 7.SMOTE-Based Class Balancing
- 8.Machine Learning Model Training
- 9.Model Performance Evaluation
- 10.Hyperparameter Optimization
- 11.Explainable Artificial Intelligence (XAI)
- 12.Web Application Deployment

Each phase was executed systematically to develop a reliable machine learning solution capable of accurately identifying hazardous Near-Earth Objects.

Methodology Description

1. Dataset Collection

The Near-Earth Object dataset provided by NASA was imported into the Google Colab environment and examined before initiating the machine learning workflow.

2. Data Preparation

The dataset was prepared by verifying missing values, checking duplicate entries, correcting data types, encoding categorical variables, and standardizing numerical features to ensure consistency during model training.

3. Exploratory Data Analysis (EDA)

Exploratory analysis was carried out using statistical summaries and visualizations to study feature distributions, identify correlations, detect outliers, and understand the balance between hazardous and non-hazardous asteroid classes.

4. Feature Engineering

Relevant predictor variables were selected and transformed into a suitable format for machine learning. Additional preprocessing techniques, including feature scaling, were applied wherever required to improve model efficiency.

5. Model Training

Multiple supervised classification algorithms, namely Logistic Regression, Decision Tree, Random Forest, Gradient Boosting, and XGBoost, were implemented and trained using the processed dataset.

6. Model Performance Evaluation

The trained models were compared using several performance indicators, including Accuracy, Precision, Recall, F1-Score, ROC-AUC Score, and Confusion Matrix, to determine the most effective classifier.

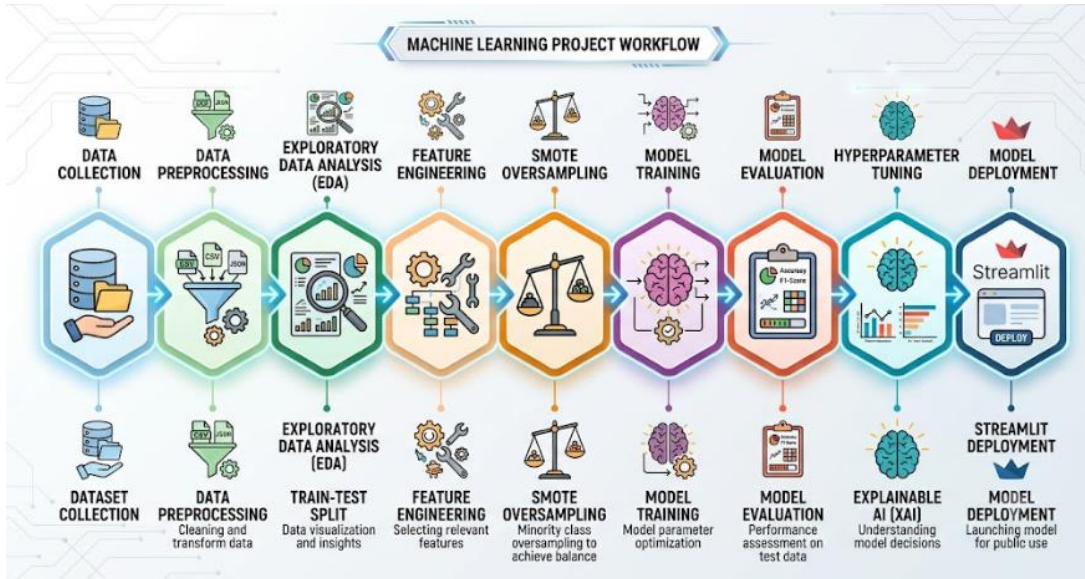
7. Explainable Artificial Intelligence (XAI)

Feature Importance analysis and SHAP (SHapley Additive Explanations) were employed to interpret the predictions of the selected model and understand the contribution of each input feature.

8. Deployment

The optimized Random Forest model was integrated into a Streamlit-based web application, allowing users to enter asteroid information and obtain real-time hazard predictions through an interactive interface

Figure 5: Overall methodology followed for the Hazardous Asteroid Prediction system.



DATA PREPROCESSING

Introduction

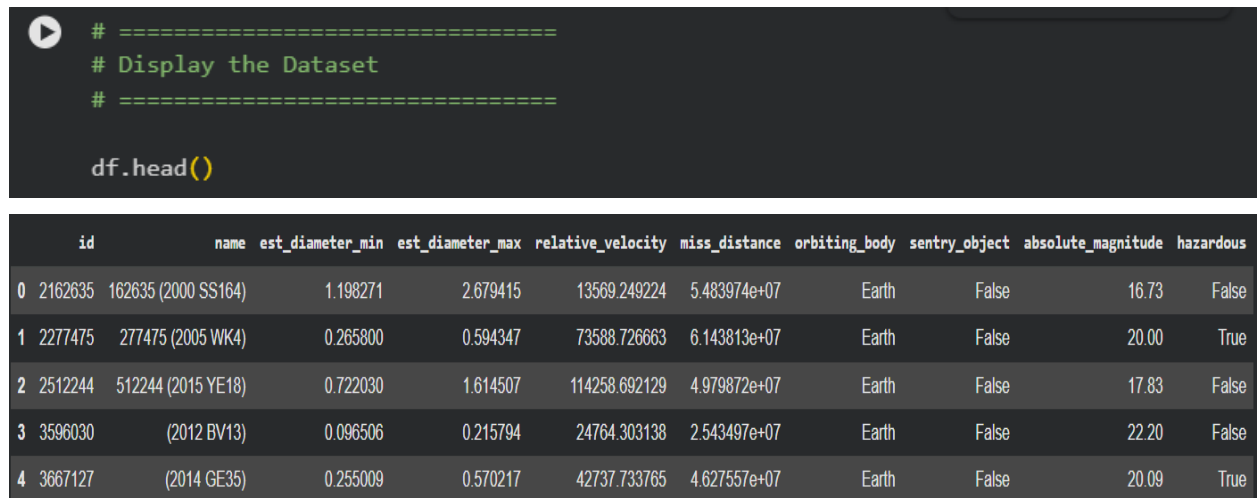
Data preprocessing is an essential step in the machine learning workflow, as the quality of the input data has a direct impact on the accuracy and reliability of the predictive model. Real-world datasets often contain issues such as missing information, duplicate entries, inconsistent data formats, categorical attributes, and imbalanced class distributions, all of which must be addressed before developing machine learning models.

For this project, the Near-Earth Object (NEO) dataset was systematically prepared to improve data consistency and make it suitable for machine learning analysis. The preprocessing process involved examining the dataset structure, checking for missing values and duplicate records, encoding categorical features, selecting relevant variables, standardizing numerical attributes, and balancing the target classes using the Synthetic Minority Over-sampling Technique (SMOTE).

1. Dataset Inspection

The dataset was initially examined to understand its structure, identify data types, and verify the number of records and attributes. This step helped identify potential issues before further preprocessing.

Figure 6: Preview of the dataset before preprocessing.



```
# =====  
# Display the Dataset  
# =====  
  
df.head()
```

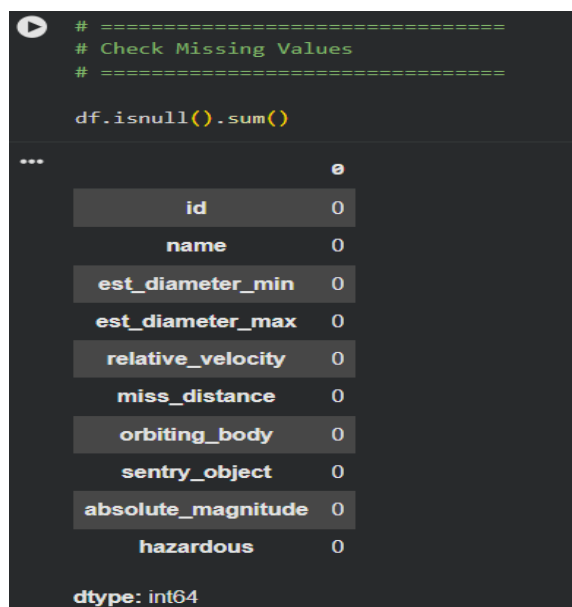
	id	name	est_diameter_min	est_diameter_max	relative_velocity	miss_distance	orbiting_body	sentry_object	absolute_magnitude	hazardous
0	2162635	162635 (2000 SS164)	1.198271	2.679415	13569.249224	5.483974e+07	Earth	False	16.73	False
1	2277475	277475 (2005 WK4)	0.265800	0.594347	73588.726663	6.143813e+07	Earth	False	20.00	True
2	2512244	512244 (2015 YE18)	0.722030	1.614507	114258.692129	4.979872e+07	Earth	False	17.83	False
3	3596030	(2012 BV13)	0.096506	0.215794	24764.303138	2.543497e+07	Earth	False	22.20	False
4	3667127	(2014 GE35)	0.255009	0.570217	42737.733765	4.627557e+07	Earth	False	20.09	True

2. Missing Value Analysis

The dataset was checked for missing values to ensure data completeness. Missing values can negatively affect the learning process and reduce prediction accuracy if not handled appropriately.

The analysis showed whether any features contained null or missing values.

Figure 7: Missing value analysis for all dataset features



```
# =====  
# Check Missing Values  
# =====  
  
df.isnull().sum()  
  
...  
0  
id 0  
name 0  
est_diameter_min 0  
est_diameter_max 0  
relative_velocity 0  
miss_distance 0  
orbiting_body 0  
sentry_object 0  
absolute_magnitude 0  
hazardous 0  
  
dtype: int64
```

3. Duplicate Record Detection

Duplicate records can introduce bias into machine learning models by giving excessive importance to repeated observations. Therefore, the dataset was checked for duplicate entries.

Any duplicate records identified were removed before proceeding to model development.

Figure 8: Duplicate record analysis.

```
# =====  
# Check Duplicate Records  
# =====  
  
print("Duplicate Rows:", df.duplicated().sum())  
  
... Duplicate Rows: 0
```

4. Data Type Verification

The data types of all features were verified to ensure that numerical and categorical variables were correctly identified before preprocessing.

Figure 9: Verification of feature data types.

```
# =====  
# Check Data Types  
# =====  
  
df.dtypes  
  
...  
0  


|                    |         |
|--------------------|---------|
| id                 | int64   |
| name               | object  |
| est_diameter_min   | float64 |
| est_diameter_max   | float64 |
| relative_velocity  | float64 |
| miss_distance      | float64 |
| orbiting_body      | object  |
| sentry_object      | bool    |
| absolute_magnitude | float64 |
| hazardous          | bool    |

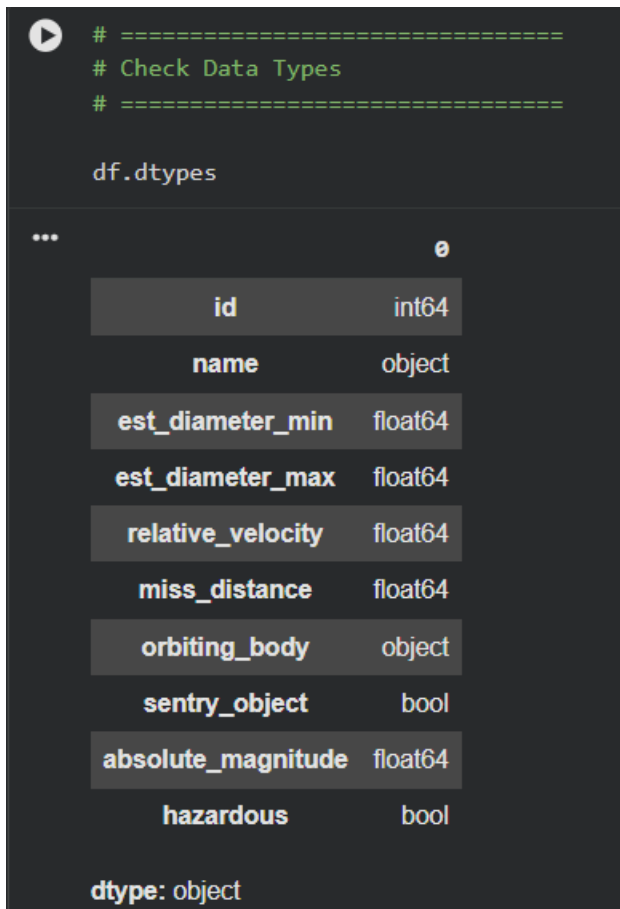
  
dtype: object
```

5. Categorical Feature Encoding

Machine learning algorithms require numerical input. Therefore, categorical variables such as **orbiting_body** and **sentry_object** were converted into numerical representations using appropriate encoding techniques.

This transformation allowed the models to process categorical information effectively while preserving the underlying relationships within the data.

Figure 10: Encoding of categorical variables.



```
# =====  
# Check Data Types  
# =====  
  
df.dtypes
```

	0
id	int64
name	object
est_diameter_min	float64
est_diameter_max	float64
relative_velocity	float64
miss_distance	float64
orbiting_body	object
sentry_object	bool
absolute_magnitude	float64
hazardous	bool

dtype: object

```

# =====
# Identify Numerical & Categorical Columns
# =====

numerical_columns = df.select_dtypes(include=['int64','float64']).columns

categorical_columns = df.select_dtypes(include=['object','bool']).columns

print("Numerical Columns:")
print(numerical_columns)

print("\nCategorical Columns:")
print(categorical_columns)

... Numerical Columns:
Index(['id', 'est_diameter_min', 'est_diameter_max', 'relative_velocity',
      'miss_distance', 'absolute_magnitude'],
      dtype='object')

Categorical Columns:
Index(['name', 'orbiting_body', 'sentry_object', 'hazardous'], dtype='object')

```

```

# =====
# Convert Boolean Target to Integer
# =====

if df['hazardous'].dtype == bool:
    df['hazardous'] = df['hazardous'].astype(int)

df['hazardous'].head()

...

```

	hazardous
0	0
1	1
2	0
3	0
4	1

```

dtype: int64

```

```

# =====
# Remove Unnecessary Columns
# =====

columns_to_drop = ['id', 'name']

existing_columns = [col for col in columns_to_drop if col in df.columns]

df = df.drop(columns=existing_columns)

print(df.head())

...

```

	est_diameter_min	est_diameter_max	relative_velocity	miss_distance	\
0	1.198271	2.679415	13569.249224	5.483974e+07	
1	0.265800	0.594347	73588.726663	6.143813e+07	
2	0.722030	1.614507	114258.692129	4.979872e+07	
3	0.096506	0.215794	24764.303138	2.543497e+07	
4	0.255009	0.570217	42737.733765	4.627557e+07	

	orbiting_body	sentry_object	absolute_magnitude	hazardous
0	Earth	False	16.73	0
1	Earth	False	20.00	1
2	Earth	False	17.83	0
3	Earth	False	22.20	0
4	Earth	False	20.09	1

```

# =====
# Encode Categorical Columns
# =====

df.select_dtypes(include='object').columns

Index(['orbiting_body'], dtype='object')

```

```

from sklearn.preprocessing import LabelEncoder

le = LabelEncoder()

for col in df.select_dtypes(include='object').columns:
    df[col] = le.fit_transform(df[col])

print("Categorical Encoding Completed")

... Categorical Encoding Completed

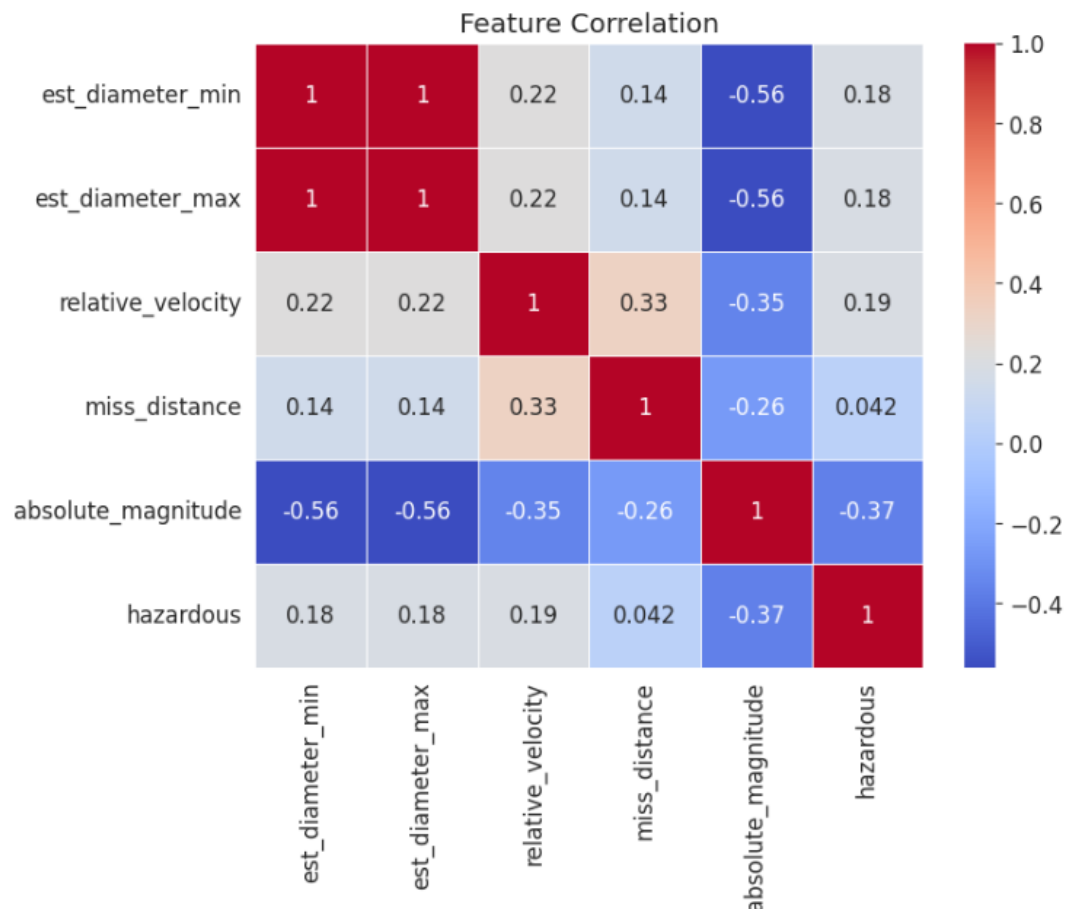
```

6. Feature Scaling

The numerical features in the dataset have different ranges and units. To prevent features with larger values from dominating the learning process, feature scaling was applied using **StandardScaler**.

Standardization transforms the features to have a mean of zero and a standard deviation of one, improving the convergence and performance of many machine learning algorithms.

Figure 11: Feature scaling



```
# =====  
# Identify Highly Correlated Features  
# =====  
  
corr_matrix = df.corr().abs()  
  
upper = corr_matrix.where(  
    np.triu(np.ones(corr_matrix.shape), k=1).astype(bool)  
)  
  
upper
```

	est_diameter_min	est_diameter_max	relative_velocity	miss_distance	absolute_magnitude	hazardous
est_diameter_min	NaN	1.0	0.221553	0.142241	0.560188	0.183363
est_diameter_max	NaN	NaN	0.221553	0.142241	0.560188	0.183363
relative_velocity	NaN	NaN	NaN	0.327169	0.353863	0.191185
miss_distance	NaN	NaN	NaN	NaN	0.264168	0.042302
absolute_magnitude	NaN	NaN	NaN	NaN	NaN	0.365267
hazardous	NaN	NaN	NaN	NaN	NaN	NaN

```
# =====  
# Display Features  
# =====  
  
print(X.head())
```

	est_diameter_min	relative_velocity	miss_distance	absolute_magnitude
0	1.198271	13569.249224	5.483974e+07	16.73
1	0.265800	73588.726663	6.143813e+07	20.00
2	0.722030	114258.692129	4.979872e+07	17.83
3	0.096506	24764.303138	2.543497e+07	22.20
4	0.255009	42737.733765	4.627557e+07	20.09

```
# =====  
# Feature Importance Using Random Forest  
# =====  
  
from sklearn.ensemble import RandomForestClassifier  
  
rf = RandomForestClassifier(  
    random_state=42  
)  
  
rf.fit(X,y)
```

...
RandomForestClassifier ⓘ ?
RandomForestClassifier(random_state=42)

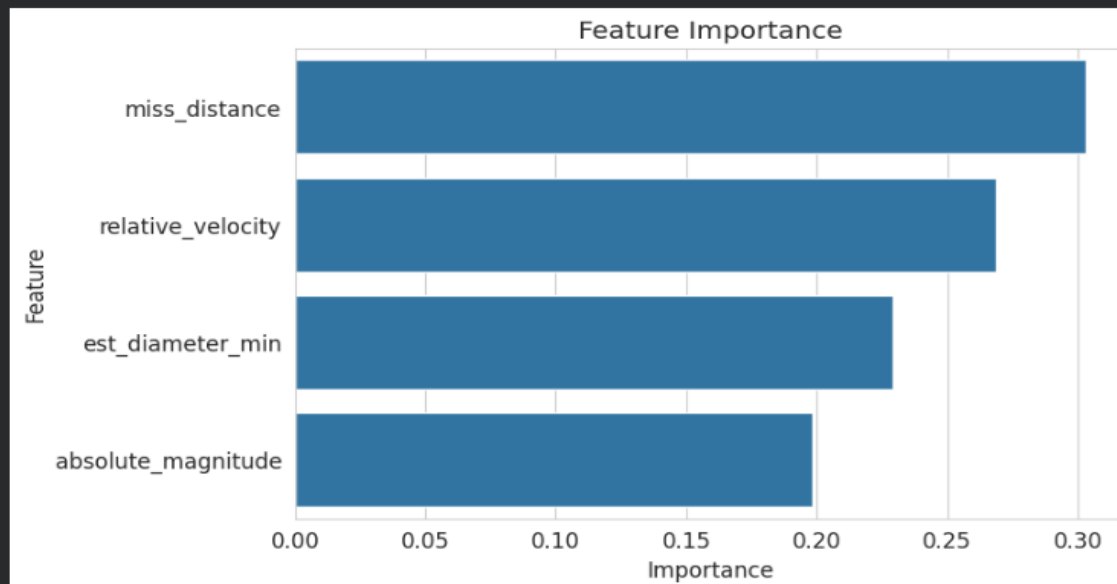
```
# =====
# Plot Feature Importance
# =====

plt.figure(figsize=(8,5))

sns.barplot(
    data=importance,
    x='Importance',
    y='Feature'
)

plt.title("Feature Importance")

plt.show()
```



```
# =====
# Verify Scaling
# =====

X_scaled.describe().T
```

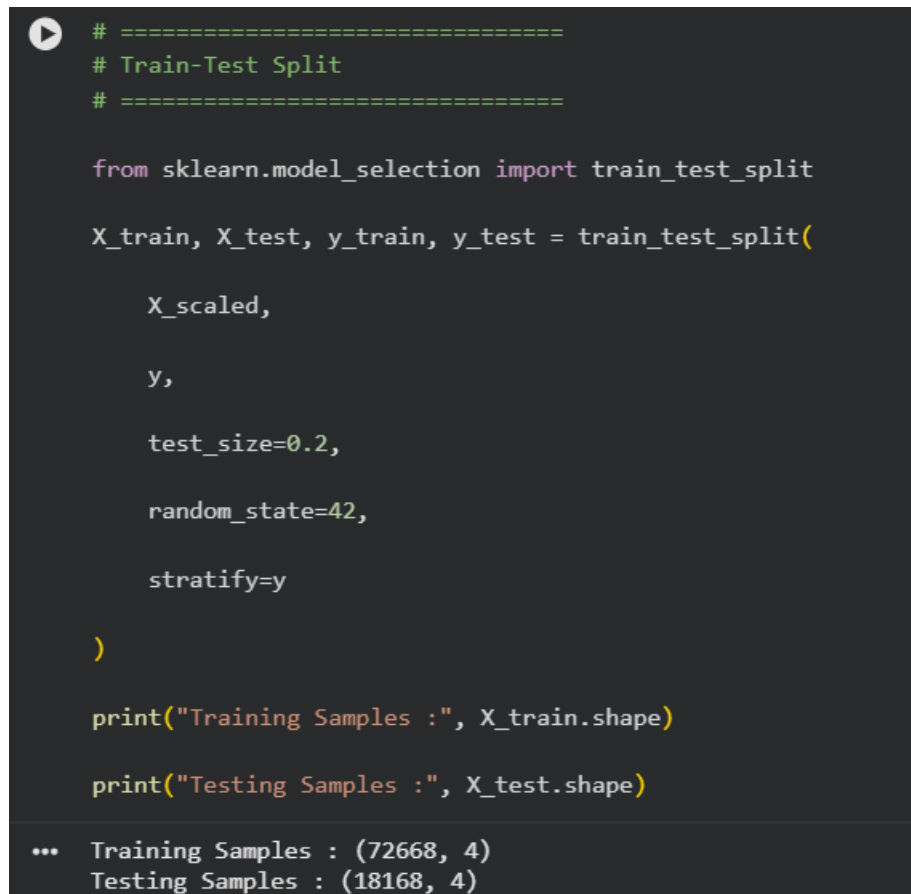
	count	mean	std	min	25%	50%	75%	max
est_diameter_min	90836.0	-6.070073e-17	1.000006	-0.424855	-0.362389	-0.264864	0.053499	126.512573
relative_velocity	90836.0	-1.742017e-16	1.000006	-1.892353	-0.768900	-0.153275	0.587380	7.469341
miss_distance	90836.0	-1.239046e-16	1.000006	-1.658014	-0.888323	0.034898	0.871623	1.688093
absolute_magnitude	90836.0	4.552555e-16	1.000006	-4.940138	-0.755719	0.059742	0.750810	3.342317

7. Train-Test Split

The preprocessed dataset was divided into training and testing sets. The training dataset was used to train the machine learning models, while the testing dataset was reserved for evaluating their performance on unseen data.

This separation ensures that the models are assessed fairly and helps prevent overfitting.

Figure 12: Training and testing dataset sizes.

The image shows a Jupyter Notebook interface with a dark background. At the top left of the code cell is a play button icon. The code is written in a light green and yellow monospace font. It imports `train_test_split` from `sklearn.model_selection` and then calls the function with `X_scaled`, `y`, `test_size=0.2`, `random_state=42`, and `stratify=y`. Below the function call, there are two `print` statements to display the shapes of the training and testing datasets. The output at the bottom of the cell shows the training dataset has 72668 samples and 4 features, while the testing dataset has 18168 samples and 4 features.

```
# =====  
# Train-Test Split  
# =====  
  
from sklearn.model_selection import train_test_split  
  
X_train, X_test, y_train, y_test = train_test_split(  
    X_scaled,  
    y,  
    test_size=0.2,  
    random_state=42,  
    stratify=y  
)  
  
print("Training Samples :", X_train.shape)  
print("Testing Samples :", X_test.shape)  
  
... Training Samples : (72668, 4)  
    Testing Samples : (18168, 4)
```

8. Handling Class Imbalance Using SMOTE

The target variable exhibited class imbalance, with significantly fewer hazardous asteroid samples than non-hazardous ones. Such imbalance can cause machine learning models to become biased toward the majority class.

To overcome this issue, the **Synthetic Minority Over-sampling Technique (SMOTE)** was applied to generate synthetic samples for the minority class, resulting in a balanced training dataset.

Figure 13: Class distribution before applying SMOTE .

```
# =====  
# Check Class Distribution Before SMOTE  
# =====  
  
print(y_train.value_counts())  
  
... hazardous  
0    65596  
1     7072  
Name: count, dtype: int64
```

Figure 14: Balanced class distribution after applying SMOTE.

```
# =====  
# Apply SMOTE  
# =====  
  
from imblearn.over_sampling import SMOTE  
  
smote = SMOTE(random_state=42)  
  
X_train_smote, y_train_smote = smote.fit_resample(  
    X_train,  
    y_train  
)  
  
# =====  
# Verify SMOTE  
# =====  
  
print(y_train_smote.value_counts())  
  
... hazardous  
0    65596  
1    65596  
Name: count, dtype: int64
```

We use SMOTE to make dataset from imbalanced to balanced. This SMOTE (Synthetic minority over-sampling technique)

Instead of simply duplicating minority-class samples, SMOTE creates new synthetic examples by interpolating between existing minority-class data points. This can help improve the performance of classification models on the minority class.

```
# =====
# Plot After SMOTE
# =====

plt.figure(figsize=(6,5))

sns.countplot(x=y_train_smote)

plt.title("Class Distribution After SMOTE")

plt.show()
```



Observation

The preprocessing stage successfully transformed the raw dataset into a clean, structured, and balanced dataset suitable for machine learning. Missing values and duplicate records were addressed, categorical variables were encoded, numerical features were standardized, and class imbalance was corrected using SMOTE. These preprocessing steps significantly improved the quality of the data and established a strong foundation for accurate model training and evaluation.

EXPLORATORY DATA ANALYSIS (EDA)

Introduction

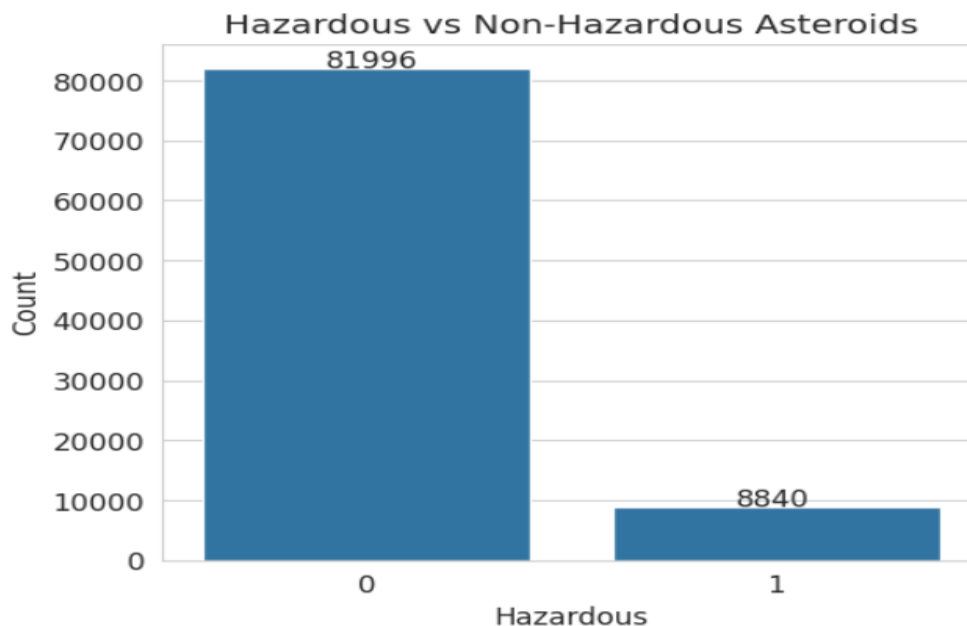
Exploratory Data Analysis (EDA) is an essential phase in the machine learning workflow that focuses on understanding the characteristics, structure, and relationships within the dataset before model development. By using statistical summaries and graphical visualizations, EDA helps identify patterns, trends, outliers, and feature distributions that may influence model performance. In this project, Exploratory Data Analysis was performed on the Near-Earth Object (NEO) dataset to gain insights into the physical and orbital characteristics of asteroids. Various visualization techniques were employed to examine the distribution of features, analyze relationships between variables, detect outliers, and understand the class distribution of hazardous and non-hazardous asteroids.

The insights obtained during this phase guided feature engineering, data preprocessing, and the selection of suitable machine learning algorithms.

1. Class Distribution Analysis

The first step in the exploratory analysis was to examine the distribution of the target variable (**hazardous**). Understanding the proportion of hazardous and non-hazardous asteroids is important because an imbalanced dataset can negatively affect classification performance.

Figure 15: Distribution of Hazardous and Non-Hazardous Astero



The visualization revealed that the number of non-hazardous asteroids is significantly higher than

the number of hazardous asteroids, confirming the presence of class imbalance.

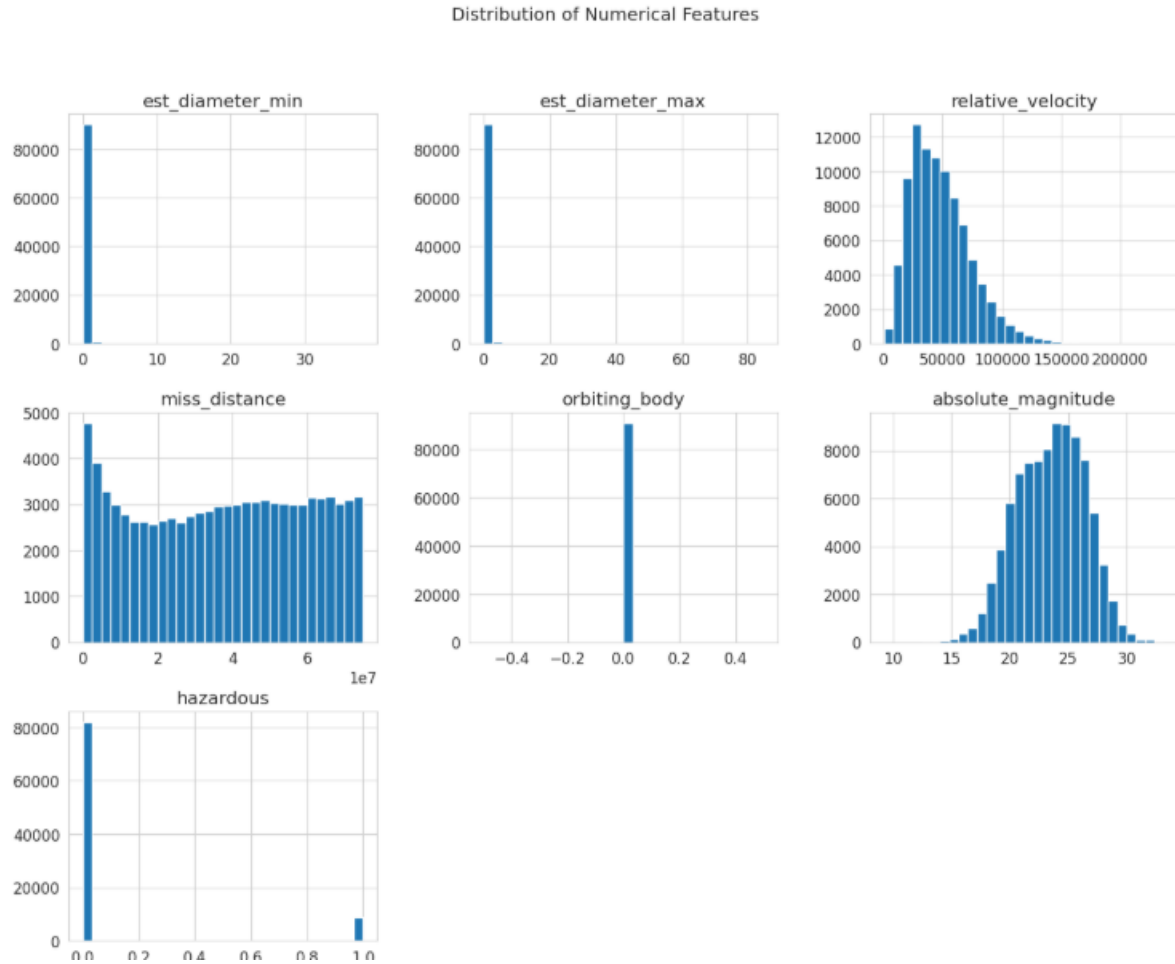
This observation justified the use of the **Synthetic Minority Over-sampling Technique (SMOTE)** during data preprocessing to balance the dataset before model training.

2. Distribution of Numerical Features

Histograms were generated to study the distribution of numerical features such as estimated diameter, relative velocity, miss distance, and absolute magnitude.

The analysis showed that several numerical variables exhibit skewed distributions, indicating the presence of extreme values and varying scales among different features. Understanding these distributions helped determine the need for feature scaling before model training.

Figure 16: Distribution of Numerical Features.



3. Outlier Detection

Boxplots were used to identify potential outliers in the numerical features. Outliers represent observations that deviate significantly from the majority of the data and may influence machine

learning algorithms.

The analysis indicated that certain features, particularly relative velocity and miss distance, contain extreme values. Since these observations represent genuine astronomical measurements rather than data entry errors, they were retained for further analysis.

Figure 17: Outlier Detection Using Boxplots.



Observation

The exploratory analysis revealed that the dataset contains an imbalanced target distribution and several skewed numerical features with valid extreme observations. These findings highlighted the importance of applying feature scaling and SMOTE to improve the robustness and predictive performance of the machine learning models.

CORRELATION ANALYSIS

Introduction

Correlation analysis measures the strength and direction of the relationship between numerical variables. Understanding these relationships helps identify influential features, detect multicollinearity, and support feature selection for machine learning.

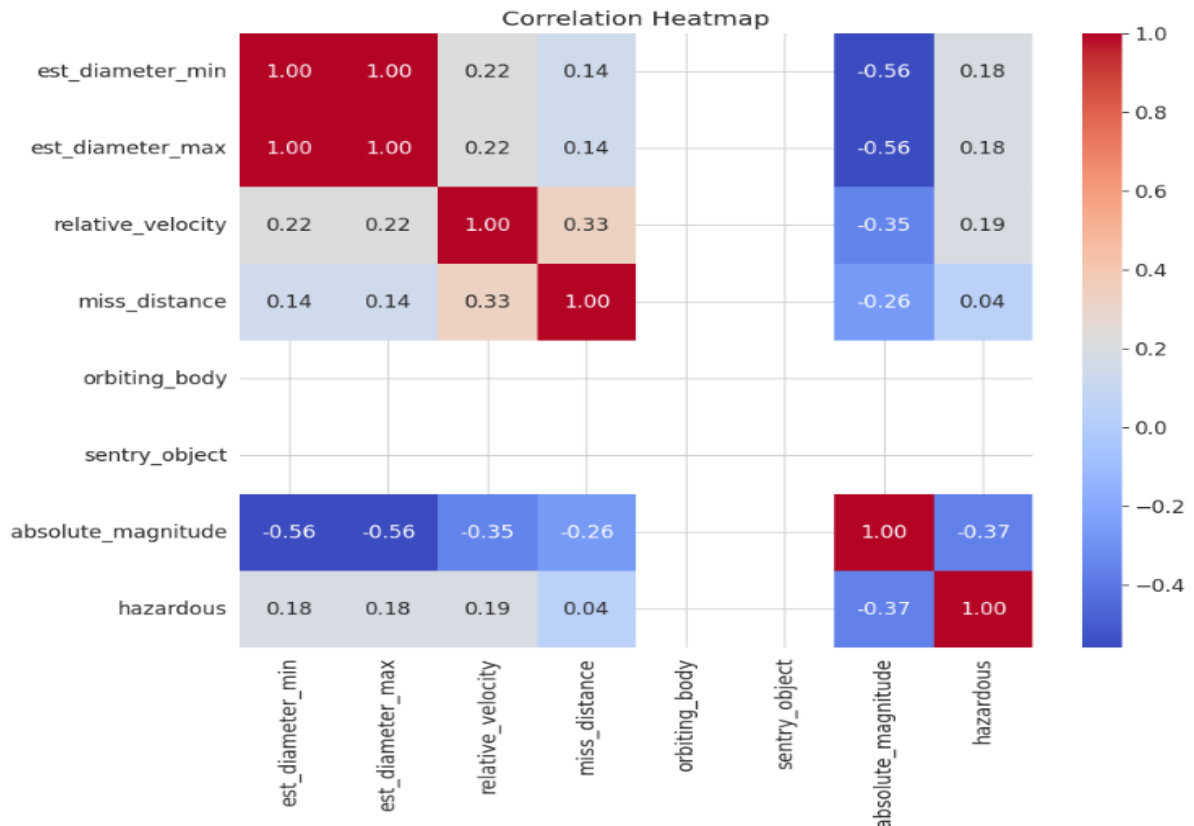
A correlation heatmap was generated to visualize the relationships among the numerical attributes of the Near-Earth Object dataset.

Correlation Heatmap

The correlation matrix illustrates the degree of linear association between pairs of numerical variables. Positive correlation values indicate that two variables tend to increase together, while negative values indicate an inverse relationship.

The analysis revealed that some features exhibit moderate correlations, whereas others are relatively independent. These insights helped identify the most informative variables for hazardous asteroid prediction.

Figure 18: Correlation Matrix of Numerical Features.

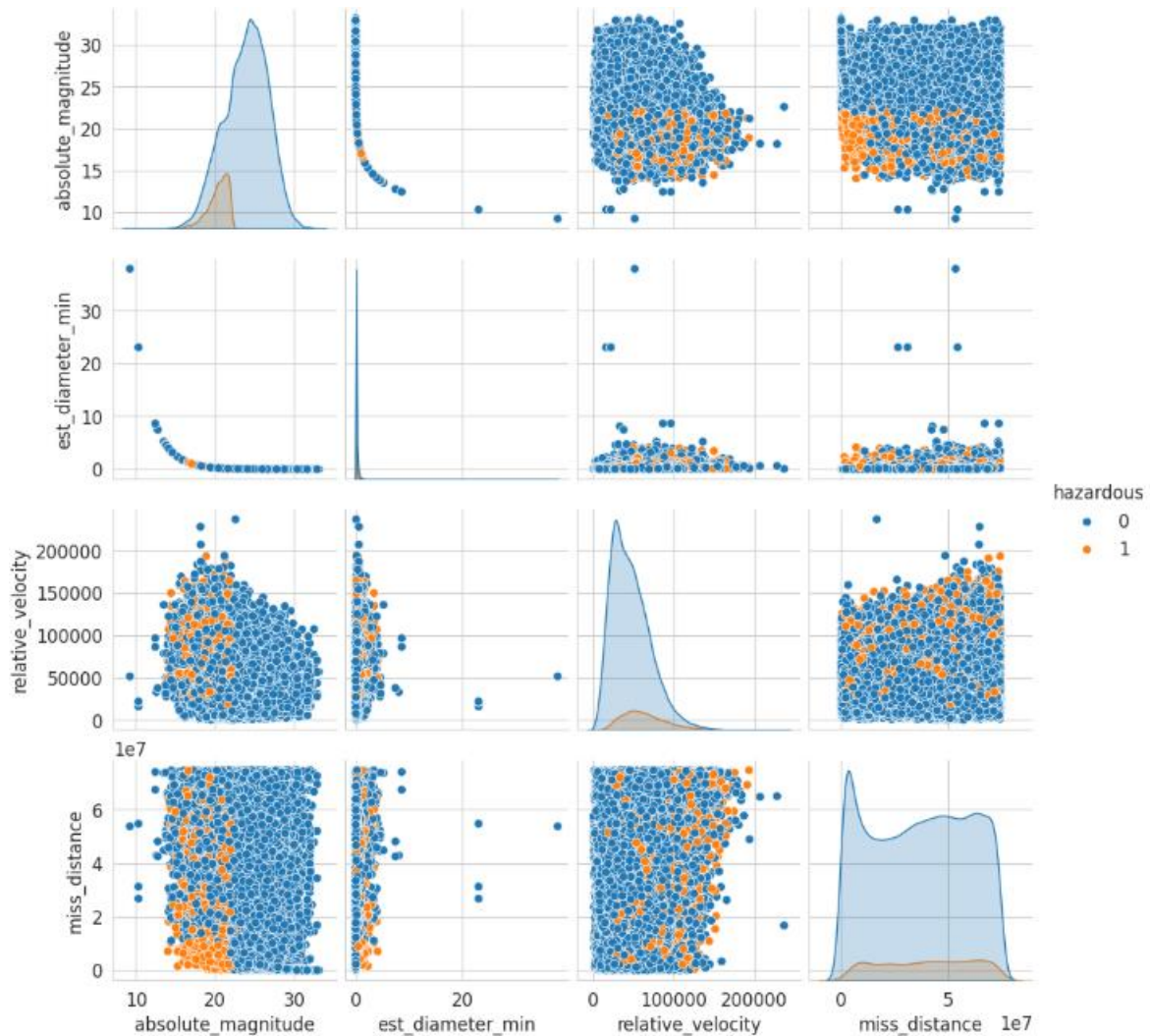


Pairwise Feature Relationships

If a pairplot was generated during EDA, include it here.

The pairplot provides a graphical representation of relationships between numerical variables and highlights how hazardous and non-hazardous asteroids are distributed across different feature combinations.

Figure 19: Pairwise Relationships Among Numerical Features.



Observation

The correlation analysis indicated that asteroid diameter, absolute magnitude, relative velocity, and miss distance contribute differently to the prediction task. These insights supported feature engineering and the selection of appropriate machine learning models.

FEATURE ENGINEERING

Introduction

Feature engineering is the process of transforming raw data into meaningful input variables that improve machine learning performance. Well-designed features enable models to learn important patterns more effectively and produce more accurate predictions.

In this project, feature engineering involved selecting relevant variables, encoding categorical features, standardizing numerical attributes, and preparing the final feature matrix for model training.

Feature Selection

The following features were selected because they directly describe the physical and orbital properties of Near-Earth Objects and contribute to hazard prediction:

- Estimated Minimum Diameter
- Estimated Maximum Diameter
- Relative Velocity
- Miss Distance
- Orbiting Body
- Sentry Object
- Absolute Magnitude

The target variable used for classification was **hazardous**.

Feature Encoding

Categorical variables such as **orbiting_body** and **sentry_object** were converted into numerical values using suitable encoding techniques. This transformation enabled the machine learning algorithms to process the categorical information effectively.

Figure 20: Categorical Feature Encoding.

```
from sklearn.preprocessing import LabelEncoder

le = LabelEncoder()

for col in df.select_dtypes(include='object').columns:
    df[col] = le.fit_transform(df[col])

print("Categorical Encoding Completed")

... Categorical Encoding Completed
```

Feature Scaling

The numerical features were standardized using **StandardScaler** to ensure that all variables contributed equally during model training.

Standardization transformed each feature to have a mean of zero and a standard deviation of one, improving convergence and model performance.

Figure 21: Standardized Feature Matrix.

```
# =====  
# Standardize Features  
# =====  
  
from sklearn.preprocessing import StandardScaler  
  
scaler = StandardScaler()  
  
X_scaled = scaler.fit_transform(X)  
  
X_scaled = pd.DataFrame(  
    X_scaled,  
    columns=X.columns  
)  
  
X_scaled.head()
```

...	est_diameter_min	relative_velocity	miss_distance	absolute_magnitude
0	3.587284	-1.363913	0.795153	-2.348632
1	0.463529	1.009040	1.090357	-1.218735
2	1.991887	2.616983	0.569624	-1.968544
3	-0.103601	-0.921301	-0.520384	-0.458560
4	0.427378	-0.210697	0.412001	-1.187637

Final Feature Matrix

The final processed dataset consisted of encoded categorical variables, standardized numerical features, and the target variable, making it suitable for machine learning model development.

Figure 22: Final Feature Matrix Used for Model Training.

	count	mean	std	min	25%	50%	75%	max
est_diameter_min	90836.0	-6.070073e-17	1.000006	-0.424855	-0.362389	-0.264864	0.053499	126.512573
relative_velocity	90836.0	-1.742017e-16	1.000006	-1.892353	-0.768900	-0.153275	0.587380	7.469341
miss_distance	90836.0	-1.239046e-16	1.000006	-1.658014	-0.888323	0.034898	0.871623	1.688093
absolute_magnitude	90836.0	4.552555e-16	1.000006	-4.940138	-0.755719	0.059742	0.750810	3.342317

MACHINE LEARNING MODEL DEVELOPMENT

Introduction

After completing data preprocessing and feature engineering, the processed dataset was used to develop supervised machine learning models for predicting hazardous asteroids. The objective was to train multiple classification algorithms, compare their predictive performance, and identify the most suitable model for deployment.

The dataset was divided into training and testing subsets to evaluate the models on unseen data. Since the target variable exhibited class imbalance, the Synthetic Minority Over-sampling Technique (SMOTE) was applied to the training dataset. SMOTE generated synthetic samples for the minority class, ensuring a balanced training set and improving the models' ability to identify hazardous asteroids.

Five supervised classification algorithms were implemented and evaluated:

- Logistic Regression
- Decision Tree Classifier
- Random Forest Classifier
- Gradient Boosting Classifier
- XGBoost Classifier

Each model was trained using the balanced training dataset and evaluated using Accuracy, Precision, Recall, F1-Score, and ROC-AUC metrics.

1. Train-Test Split

The processed dataset was divided into training and testing sets using an **80:20 ratio**. The training data were used for model learning, while the testing data were reserved for evaluating the models on previously unseen observations.

Figure 23: Training and testing dataset dimensions.

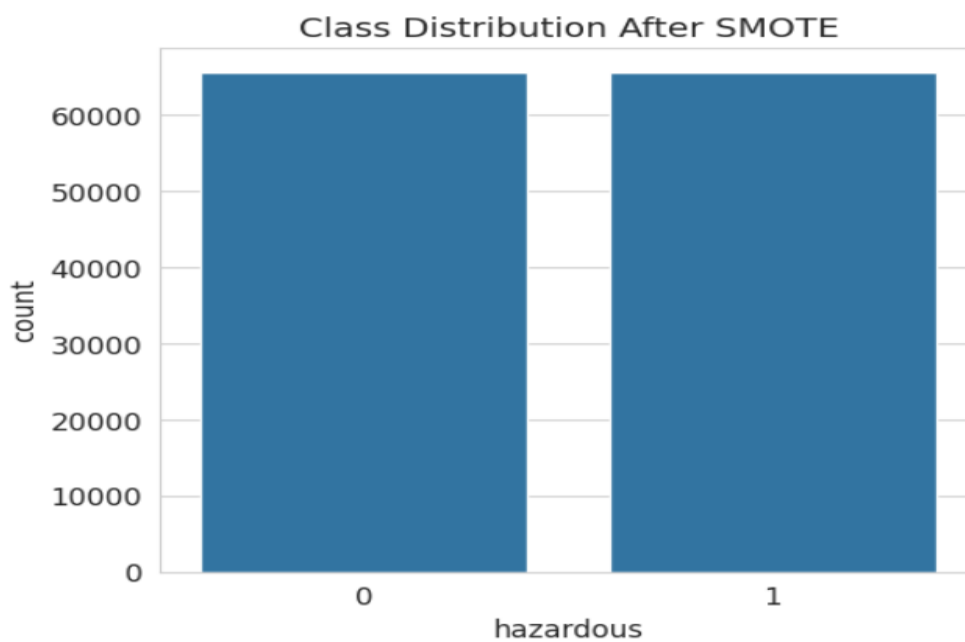
```
# =====  
# Train-Test Split  
# =====  
  
from sklearn.model_selection import train_test_split  
  
X_train, X_test, y_train, y_test = train_test_split(  
    X_scaled,  
    y,  
    test_size=0.2,  
    random_state=42,  
    stratify=y  
)  
  
print("Training Samples :", X_train.shape)  
print("Testing Samples :", X_test.shape)  
... Training Samples : (72668, 4)  
    Testing Samples : (18168, 4)
```

2. Class Balancing Using SMOTE

To overcome the imbalance between hazardous and non-hazardous asteroid classes, the **Synthetic Minority Over-sampling Technique (SMOTE)** was applied to the training dataset. This technique generated synthetic samples for the minority class, resulting in a balanced class.

Balancing the training data improved the learning process and enhanced the models' ability to correctly classify hazardous asteroids.

Figure 24: Class distribution after applying SMOTE.



3. Machine Learning Models

The following supervised learning algorithms were implemented:

Logistic Regression

Logistic Regression is a linear classification algorithm that estimates the probability of a binary outcome using the logistic (sigmoid) function. It served as the baseline model for comparison.

Decision Tree Classifier

The Decision Tree Classifier builds a hierarchical tree structure by recursively splitting the dataset based on feature values. It is easy to interpret and captures non-linear decision boundaries.

Random Forest Classifier

Random Forest is an ensemble learning algorithm that constructs multiple decision trees and combines their predictions through majority voting. This approach improves predictive accuracy and reduces overfitting.

Gradient Boosting Classifier

Gradient Boosting builds a sequence of weak learners, where each new model corrects the errors of the previous one. It is capable of learning complex relationships within the data.

XGBoost Classifier

Extreme Gradient Boosting (XGBoost) is an optimized gradient boosting framework that offers high computational efficiency, regularization, and strong predictive performance.

Observation

Multiple machine learning algorithms were implemented to evaluate different learning approaches. Comparing several models ensured that the final prediction system was based on the algorithm with the best overall performance rather than selecting a model without empirical evaluation.

MODEL TRAINING AND PERFORMANCE COMPARISON

Introduction

Each machine learning algorithm was trained using the balanced training dataset and evaluated on the testing dataset. The evaluation was performed using multiple classification metrics to provide a comprehensive assessment of each model's predictive capability.

The following metrics were used:

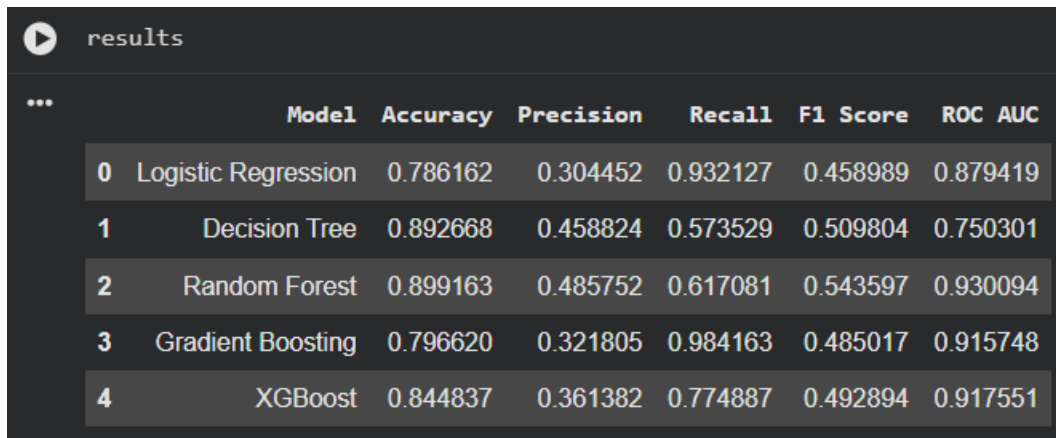
- Accuracy
- Precision
- Recall
- F1-Score
- ROC-AUC Score

Using multiple metrics ensures that the evaluation considers not only overall accuracy but also the ability to correctly identify hazardous asteroids.

Performance Comparison

The results obtained from the five machine learning models are summarized below.

Figure 25: Performance comparison of all machine learning models.



...	Model	Accuracy	Precision	Recall	F1 Score	ROC AUC
0	Logistic Regression	0.786162	0.304452	0.932127	0.458989	0.879419
1	Decision Tree	0.892668	0.458824	0.573529	0.509804	0.750301
2	Random Forest	0.899163	0.485752	0.617081	0.543597	0.930094
3	Gradient Boosting	0.796620	0.321805	0.984163	0.485017	0.915748
4	XGBoost	0.844837	0.361382	0.774887	0.492894	0.917551

Analysis

The comparison indicates that each model has different strengths. Logistic Regression and Gradient Boosting achieved high recall, demonstrating their ability to identify hazardous asteroids. Decision Tree provided strong overall accuracy but was more susceptible to overfitting.

The Random Forest Classifier achieved the best balance between Accuracy, Precision, Recall, F1-Score, and ROC-AUC. Its ensemble learning strategy produced robust predictions while reducing

variance and overfitting.

Based on this comprehensive evaluation, the Random Forest model was selected as the final prediction model for deployment.

Observation

The comparative evaluation confirmed that Random Forest consistently outperformed the other algorithms across multiple performance metrics, making it the most suitable choice for hazardous asteroid prediction.

FINAL MODEL TRAINING

Introduction

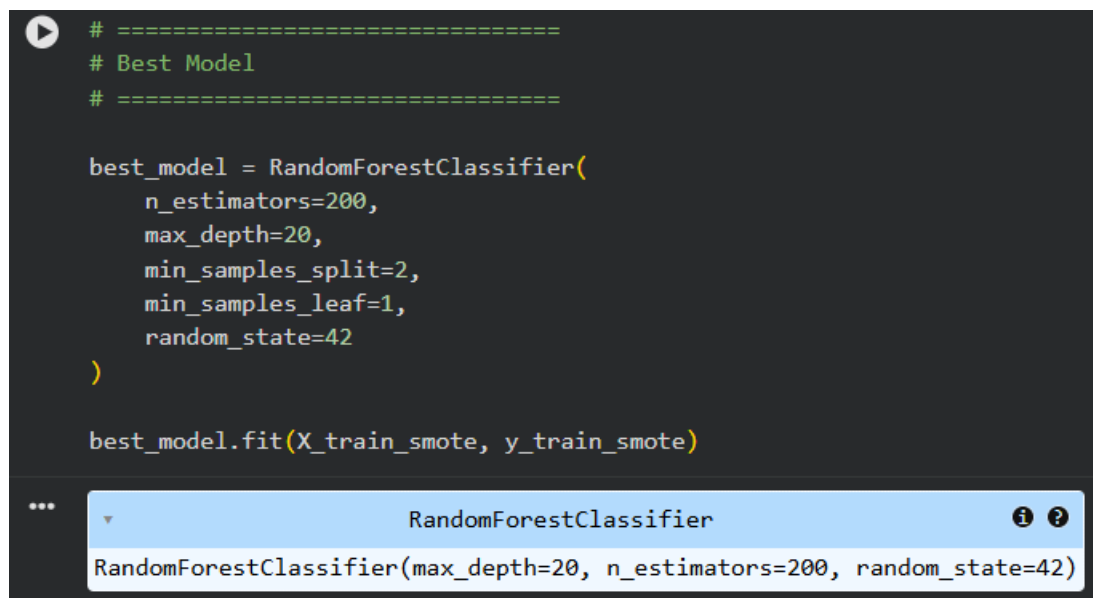
After identifying Random Forest as the best-performing algorithm, the model was retrained using the optimized hyperparameters to maximize predictive performance.

The following hyperparameters were used:

- Number of Trees (n_estimators) = 200
- Maximum Depth (max_depth) = 20
- Minimum Samples Split (min_samples_split) = 2
- Minimum Samples per Leaf (min_samples_leaf) = 1
- Random State = 42

The optimized Random Forest model was trained using the balanced training dataset generated through SMOTE.

Figure 26: Training the final Random Forest model.



```
# =====  
# Best Model  
# =====  
  
best_model = RandomForestClassifier(  
    n_estimators=200,  
    max_depth=20,  
    min_samples_split=2,  
    min_samples_leaf=1,  
    random_state=42  
)  
  
best_model.fit(X_train_smote, y_train_smote)
```

RandomForestClassifier

RandomForestClassifier(max_depth=20, n_estimators=200, random_state=42)

MODEL EVALUATION

Introduction

Model evaluation is a crucial phase in the machine learning workflow, as it measures how effectively a trained model performs on unseen data. After comparing multiple classification algorithms, the **Random Forest Classifier** was selected as the final model because it achieved the best overall balance across all evaluation metrics.

The model was evaluated using the testing dataset and assessed using the following performance metrics:

- Accuracy
- Precision
- Recall
- F1-Score
- Confusion Matrix
- ROC-AUC Score

These metrics provide a comprehensive understanding of the model's predictive capability and its effectiveness in identifying hazardous asteroids.

Classification Report

The classification report summarizes the predictive performance of the Random Forest model by presenting precision, recall, F1-score, and support for each class.

The results indicate that the model achieved strong overall classification performance while maintaining a good balance between correctly identifying hazardous asteroids and minimizing false predictions.

Figure 27: Classification Report of the Final Random Forest Model.

...	precision	recall	f1-score	support
0	0.98	0.82	0.89	16400
1	0.34	0.88	0.49	1768
accuracy			0.83	18168
macro avg	0.66	0.85	0.69	18168
weighted avg	0.92	0.83	0.86	18168

Confusion Matrix

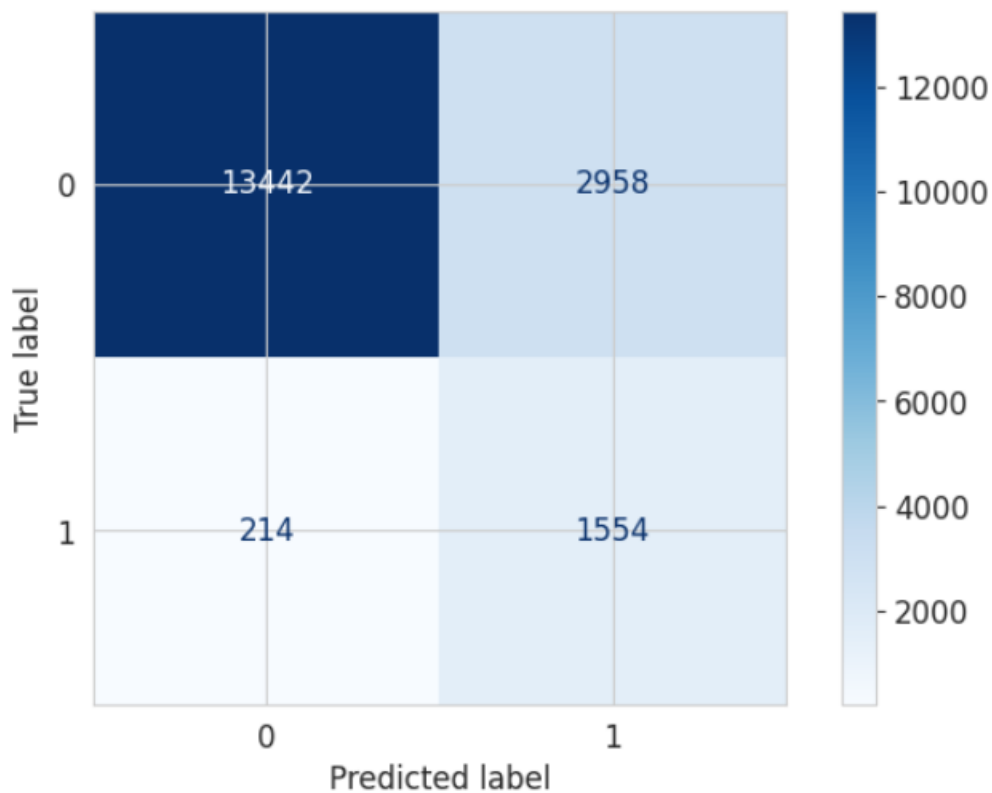
The confusion matrix provides a detailed breakdown of the model's predictions by comparing the actual class labels with the predicted labels.

It illustrates the number of:

- True Positives (Hazardous correctly predicted)
- True Negatives (Non-Hazardous correctly predicted)
- False Positives
- False Negatives

The confusion matrix demonstrates that the Random Forest model successfully classified the majority of asteroid observations while maintaining a low number of incorrect predictions.

Figure 28: Confusion Matrix of the Final Random Forest Model.



Evaluation Summary

The evaluation metrics demonstrate that the Random Forest classifier provides reliable and consistent predictions. The high ROC-AUC score indicates strong discrimination between hazardous and non-hazardous asteroids, while the balanced Precision, Recall, and F1-score confirm the model's effectiveness in handling the classification task.

EXPLAINABLE ARTIFICIAL INTELLIGENCE (XAI)

Introduction

While high predictive accuracy is important, understanding **why** a machine learning model makes a particular prediction is equally valuable, especially in scientific applications. Explainable Artificial Intelligence (XAI) enhances model transparency by identifying the features that influence predictions.

In this project, two XAI techniques were applied:

- Feature Importance
- SHAP (SHapley Additive exPlanations)

These methods provide insight into the contribution of each feature toward the model's predictions.

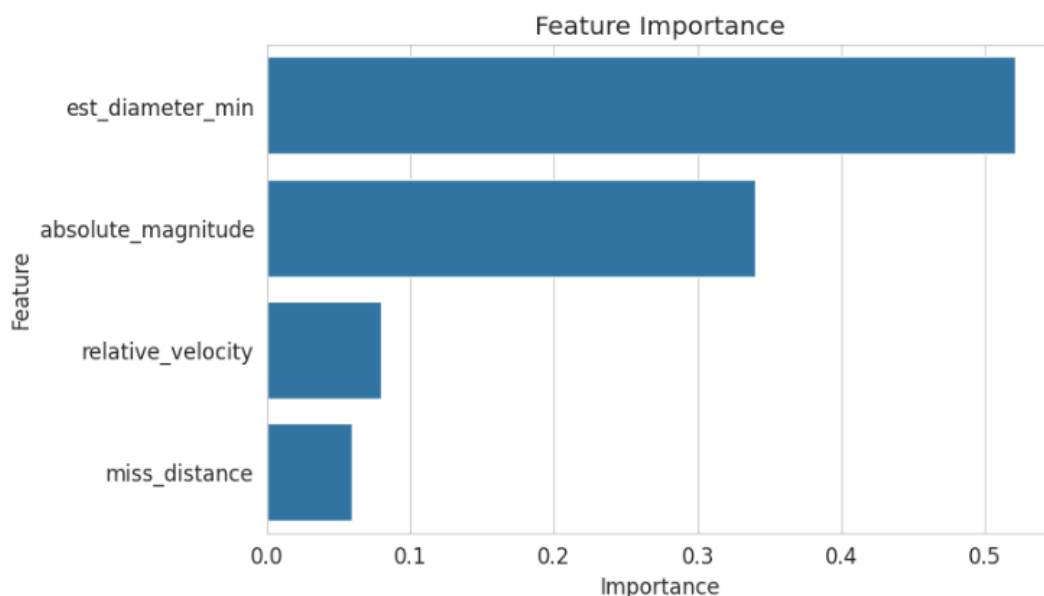
Feature Importance Analysis

Feature Importance ranks the input variables according to their influence on the Random Forest model's predictions.

The analysis revealed that features such as **absolute magnitude**, **estimated diameter**, **relative velocity**, and **miss distance** had the greatest impact on determining whether an asteroid is hazardous.

This information confirms that the model relies on scientifically meaningful characteristics when making predictions.

Figure 29: Feature Importance Ranking Generated by the Random Forest Model.



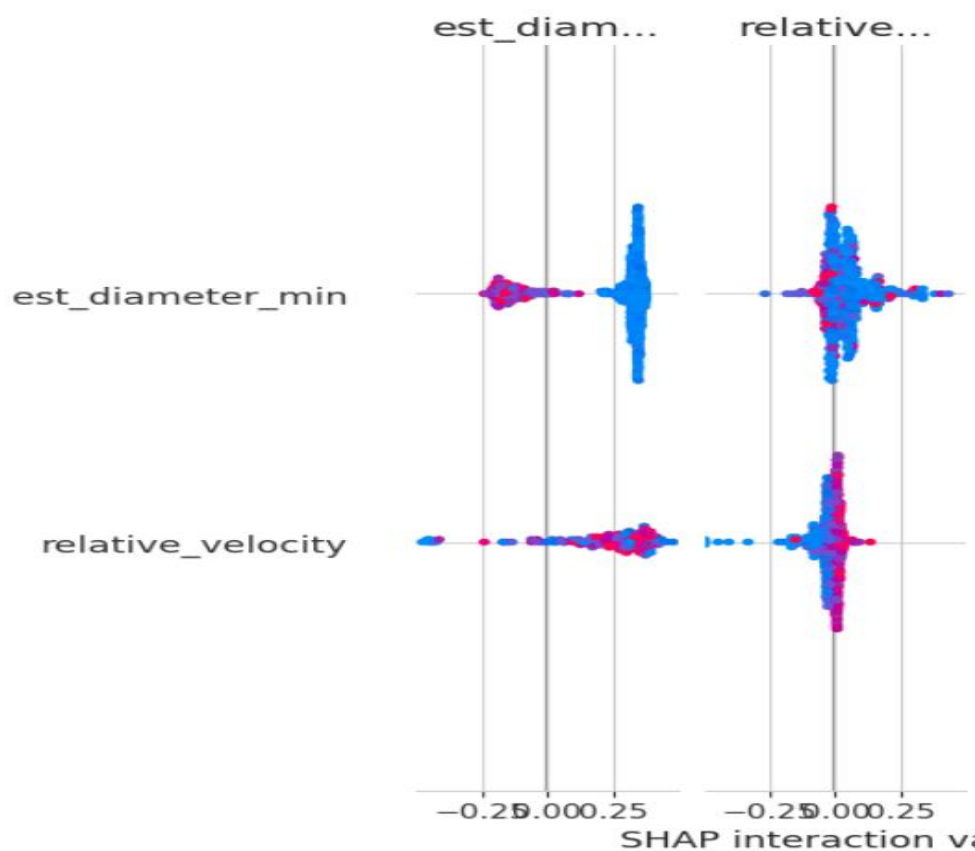
SHAP Analysis

SHAP (Shapley Additive exPlanations) is an advanced Explainable AI technique that explains individual model predictions by assigning contribution values to each feature.

The SHAP summary plot illustrates how different feature values influence the prediction of hazardous and non-hazardous asteroids. Features with higher SHAP values contribute more significantly to the final prediction.

The use of SHAP improves model transparency and increases confidence in the reliability of the prediction system.

Figure 30: SHAP Summary Plot Explaining Model Predictions.



Observation

The Explainable AI analysis confirmed that the Random Forest model bases its predictions on relevant physical and orbital characteristics rather than arbitrary patterns, thereby increasing the trustworthiness of the prediction system.

MODEL DEPLOYMENT

Introduction:

To make the developed machine learning model available for real-time use, it was deployed using the Streamlit framework. Streamlit provides a simple and efficient way to build interactive web applications using Python without requiring extensive web development knowledge.

The deployment converts the trained Random Forest classifier into an easy-to-use application that can instantly predict whether a Near-Earth Object is hazardous or non-hazardous.

Deployment Process

The deployment procedure involved the following steps:

1. Save the trained Random Forest model using the Joblib library.
2. Store the fitted StandardScaler to preprocess future input data.
3. Design an interactive user interface using Streamlit.
4. Load the saved machine learning model and scaler when the application starts.
5. Receive asteroid information entered by the user.
6. Apply feature scaling to the input using the saved StandardScaler.
7. Generate the hazard prediction using the trained model.
8. Display the prediction result on the application's interface.

Streamlit Application:

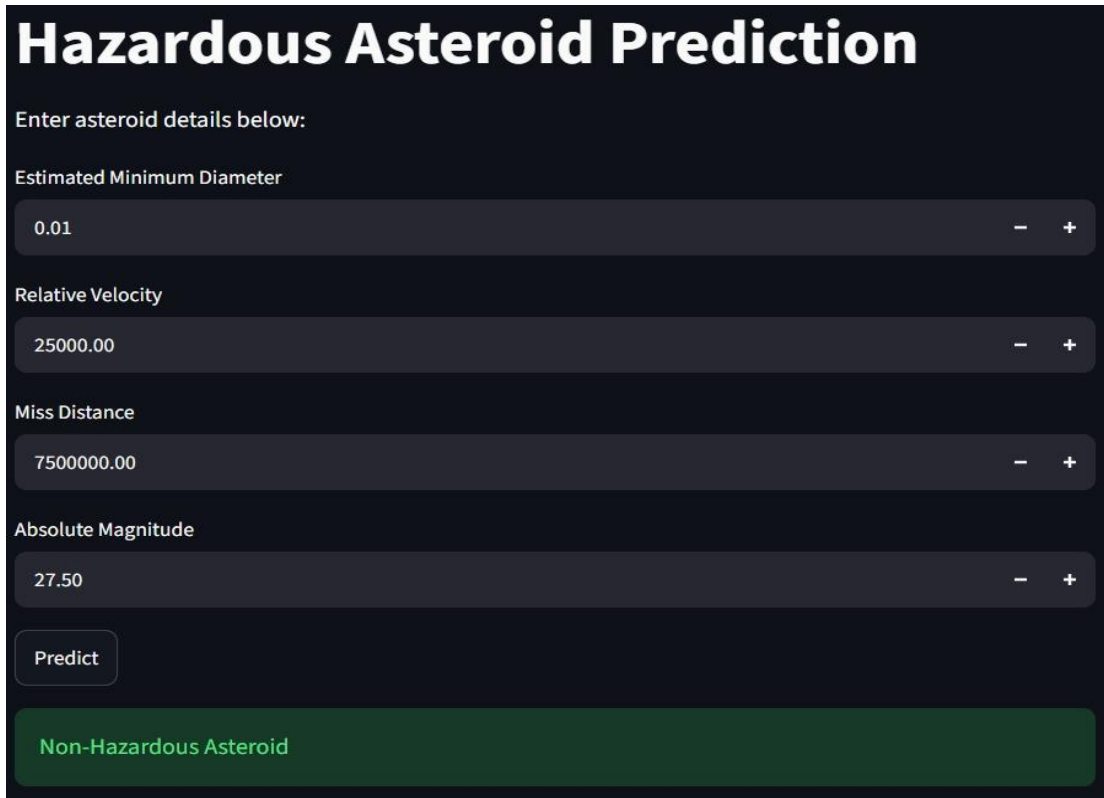
The deployed web application provides a simple and interactive interface where users can enter asteroid details such as estimated diameter, relative velocity, miss distance, absolute magnitude, orbiting body, and sentry object.

Once the Predict button is clicked, the application processes the entered values and displays the prediction result as either:

- Hazardous Asteroid
- Non-Hazardous Asteroid

The successful deployment of the application demonstrates how machine learning models can be integrated into interactive web applications, allowing users to perform real-time **asteroid hazard predictions through an accessible and user-friendly interface.**

Figure 31: Streamlit User Interface for Hazardous Asteroid Prediction.



Hazardous Asteroid Prediction

Enter asteroid details below:

Estimated Minimum Diameter

0.01 - +

Relative Velocity

25000.00 - +

Miss Distance

7500000.00 - +

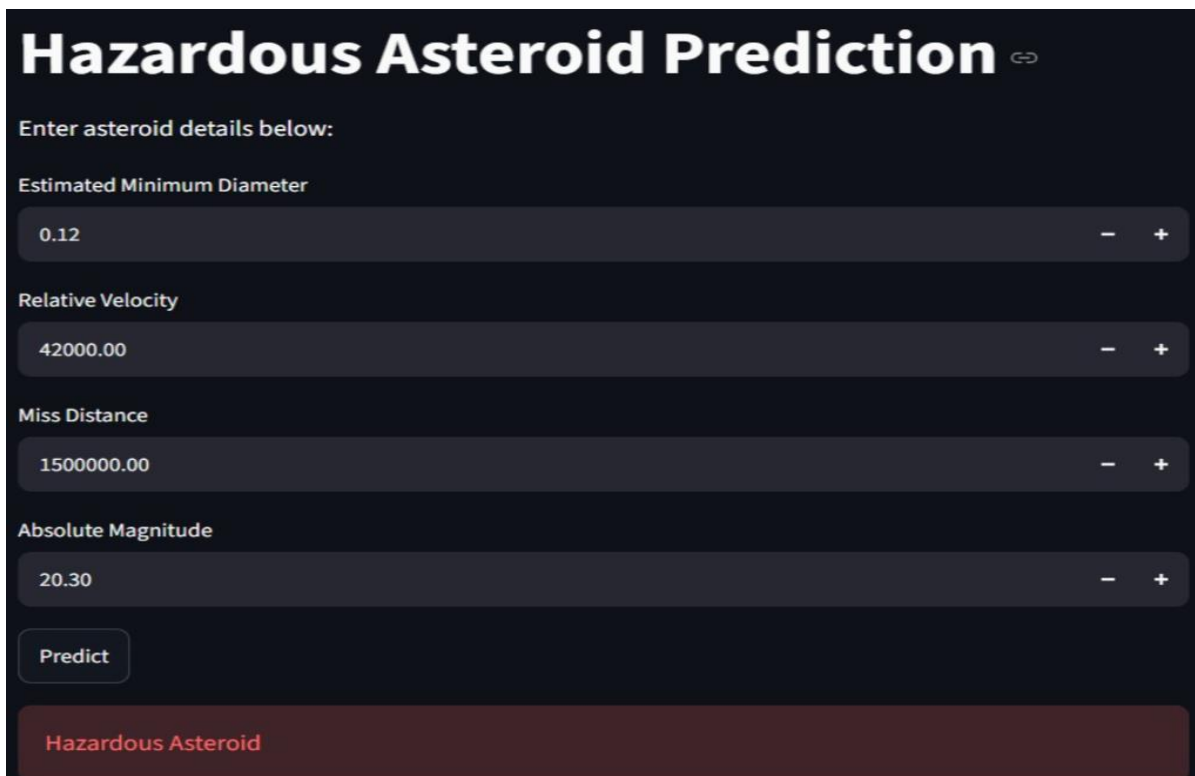
Absolute Magnitude

27.50 - +

Predict

Non-Hazardous Asteroid

Figure 32: Prediction Result Generated by the Streamlit Application.



Hazardous Asteroid Prediction [↗](#)

Enter asteroid details below:

Estimated Minimum Diameter

0.12 - +

Relative Velocity

42000.00 - +

Miss Distance

1500000.00 - +

Absolute Magnitude

20.30 - +

Predict

Hazardous Asteroid

Project Outcomes

The project successfully accomplished its intended objectives by building a complete machine learning system for predicting hazardous Near-Earth Objects using historical asteroid data.

The key outcomes of the project are summarized below:

- Successfully prepared and processed the NASA Near-Earth Object (NEO) dataset for machine learning.
- Conducted detailed Exploratory Data Analysis (EDA) to understand the dataset and identify important patterns.
- Performed feature engineering and standardized numerical features for improved model performance.
- Handled class imbalance effectively using the SMOTE oversampling technique.
- Implemented and evaluated five supervised machine learning classification models.
- Identified the Random Forest Classifier as the most effective prediction model.
- Achieved an overall accuracy of approximately 89.92% with a ROC-AUC score of 0.93.
- Improved model interpretability using Explainable Artificial Intelligence (XAI) methods.
- Successfully deployed the final prediction model as a Streamlit web application for real-time asteroid hazard prediction.

FUTURE SCOPE

Although the proposed system delivers reliable prediction results, there are several opportunities to further enhance its functionality and performance in future developments:

- Integrate the application with NASA's real-time Near-Earth Object API to obtain live asteroid data for prediction.
- Host the application on cloud platforms such as Streamlit Community Cloud or AWS to improve accessibility and scalability.
- Investigate advanced deep learning techniques to achieve higher prediction accuracy and better model performance.
- Design a real-time asteroid tracking dashboard with interactive charts and visualization features.
- Incorporate orbital trajectory analysis and simulation methods to support long-term hazard prediction.
- Expand the system to identify and classify additional categories of celestial objects beyond hazardous asteroids.

REFERENCES

The following books, research papers, official documentation, and online resources were referred to during the development and implementation of this project.

- 1.NASA. Center for Near-Earth Object Studies (CNEOS). Available at: <https://cneos.jpl.nasa.gov/>
- 2.Pedregosa, F., Varoquaux, G., Gramfort, A., et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
- 3.Chen, T., & Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*.
- 4.Lundberg, S. M., & Lee, S. I. (2017). A Unified Approach to Interpreting Model Predictions. *Advances in Neural Information Processing Systems (NeurIPS)*.
- 5.Streamlit. Official Documentation. Available at: <https://docs.streamlit.io/>
- 6.NumPy Developers. NumPy Documentation. Available at: <https://numpy.org/doc/>
- 7Pandas Development Team.Pandas Documentation: <https://pandas.pydata.org/docs/>
- 8.Matplotlib Developer Team Matplotlib Document. Available at: <https://matplotlib.org/stable/>
- 9.SHAP Developers. SHAP Documentation. Available at: <https://shap.readthedocs.io/>